

PRACTICAL-3

Aim:

To implement Heap Sort using the Heapify technique in C++ and analyze its time complexity

Input:- Array of elements
Output:- Sorted elements
Step ①:- start
Step ②:- For $i = n/2$ to 1, do
Step ③:- $largest = i$
Step ④:- $l = 2 * i$
Step ⑤:- $r = 2 * i + 1$
Step ⑥:- if $l < n$ and $arr[l] > arr[largest]$ then do
 $largest = l$
Step ⑦:- if $r < n$ and $arr[r] > arr[largest]$ then do
 $largest = r$
Step ⑧:- if $largest \neq i$ then do
Step ⑨:- swap $arr[largest]$ and $arr[i]$
Step ⑩:- for $(int i = size; i > 1; i--)$
Step ⑪:- swap $arr[i]$ and $arr[i]$
Step ⑫:- maxheapify ($arr, i - 1, 1$);
Step ⑬:- stop

Program:

1. heap sort:

```
#include <iostream>

using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;

    int left = 2 * i + 1;

    int right = 2 * i + 2;

    if(left < n && arr[left] > arr[largest])

        largest = left;

    if(right < n && arr[right] > arr[largest])

        largest = right;

    if (largest != i)
    {
        swap(arr[i], arr[largest]);

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
```

```
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }

    int main()

    {
        int arr[] = {12, 11, 13, 5, 6, 7};

        int n = 6; cout<< "Original
Array: "; for(int i = 0; i < n; i++)
        cout << arr[i] << " ";

        heapSort(arr, n);


        cout<< "\nSorted Array: ";

        for(int i = 0; i < n; i++)

        cout << arr[i] << " ";

        return 0;

    }
```

Output:

Output

Original Array: 12 11 13 5 6 7

Sorted Array: 5 6 7 11 12 13

=== Code Execution Successful ===

$\rightarrow$ for $(i = n/2; i > 1; i--) \rightarrow n$

maxHeapify(arr, n, i) $\rightarrow \log n$

for $(i = n; i > 1; i--) \rightarrow n$

swap(arr[i], arr[1]) $\rightarrow 1$

maxHeapify(arr, i-1, 1) $\rightarrow \log n$

$T(n) = n \times \log n + n \times 1 \times \log n$

$= n \log n + n \log n = 2n \log n$

$T(n) = O(n \log n)$

Time Complexity:

Best case

for $n/2$ to 1

max heapify (arr, n, i)

swap (arr[i], arr[1])

max heapify (arr, i-1, 1)

$$T(n) = O(n) + n \times O(\log n)$$

$$T(n) = O(n) + O(n \log n)$$

$$T(n) = O(n \log n)$$

Average case

for $i = n$ to 2

max heapify $\rightarrow \log n$

$$T(n) = n + n \log n$$

$$T(n) = O(n \log n)$$

Worst case

max heapify from root to last level

of heap $\Rightarrow \log_2 n$

max heapify $\rightarrow \log n$

$$T(n) = n + n \cdot \log n$$

$$T(n) = O(n \log n)$$

Conclusion:

Heap Sort was successfully implemented using the Heapify technique. The array was sorted in ascending order with $O(n \log n)$ time complexity.